Generics in Java

Generics in Java were introduced in Java 5 to allow the creation of classes, interfaces, and methods that can operate on any defined type while providing compile-time type safety. The key idea behind generics is to enable code reusability and flexibility without compromising type safety.

Type Safety in Java : Type safety means you can only store and use the correct type of data, and errors are caught at compile time, not runtime.

Without Type Safety:

```
import java.util.ArrayList;
public class Test {
  public static void main(String[] args) {
    ArrayList list = new ArrayList(); // raw type
    list.add("Vivek");
    list.add(100); // allowed
    String name = (String) list.get(1); // Runtime error
    System.out.println(name);
  }
}
```

Compiler allows mixed types

At runtime → ClassCastException Problem: No type safety

With Type Safety :

```
import java.util.ArrayList;
public class Test {
  public static void main(String[] args) {
    ArrayList<String> list = new ArrayList<>();
    list.add("Vivek");
    // list.add(100);  Compile-time error
    String name = list.get(0); // No casting needed
    System.out.println(name);
  }
}
```

- Only String allowed
- Error caught at **compile time**
- No runtime crash
- This is **Type Safety**

Type safety ensures that only the intended data type is used, preventing runtime errors by catching issues at compile time.

Type Casting in Java : Type Casting means converting one data type into another.

```
ArrayList list = new ArrayList();
list.add("Hello");
String s = (String) list.get(0); // manual casting needed
```

With Generics:

```
ArrayList<String> list = new ArrayList<>();
list.add("Hello");
String s = list.get(0); // No casting
```

Arrays : Type Safe (Arrays in Java are type-safe by default);

```
public class Test {
  public static void main(String[] args) {
     String[] arr = new String[2];
     arr[0] = "Vivek";
     // arr[1] = 100;  Compile-time error
     System.out.println(arr[0]);
  }
}
```

Array is declared as String[ ] so it can store only String so Compiler stops wrong type → Type Safety

No Type Casting Needed in Arrays:

```
public class Test {
  public static void main(String[] args) {
     String[] arr = {"A", "B", "C"};
     String s = arr[0]; // No casting needed
     System.out.println(s);} }
```

Note: Arrays are inherently type-safe and do not require casting, whereas collections achieve the same safety using generics.

ArrayList with Type-safe and No casting using Generics (Same as Array)

```
import java.util.ArrayList;
public class Test {
  public static void main(String[] args) {
     ArrayList<String> list = new ArrayList<>();
     list.add("Vivek");
     // list.add(100); Compile-time error
     String s = list.get(0); // No casting
  }
}
```

Why Generics: Compile-time safety + No ClassCastException.

Generic Class:

```
class Box<T> {
  private T value;
  public void set(T value) {
     this.value = value;
  }
  public T get() {
     return value;
  }
 public static void main(String[] args) {
        Box<Integer> intBox = new Box<>();
        intBox.set(100);
        Box<String> strBox = new Box<>();
        strBox.set("Hello");
        }
}
```

T = Type Parameter (can be any type);

Generic Method in Java: A generic method is a method that can work with any data type, using a type parameter <T>.

Basic Syntax:

```
public <T> void print(T data) {
  System.out.println(data);
}
```

<T> comes **before return type** T can be any type (String, Integer, etc.)

Example:

```
class Test {
  public static <T> void show(T value) {
     System.out.println(value);
  }
  public static void main(String[] args) {
     show("Hello");  // String
     show(100);      // Integer
     show(10.5);     // Double
  }
}
```

O/P:-

Hello

100

10.5

Same method works for multiple types → **Reusability + Type Safety**

Generic Method with Return Type:

```
class Test {
  public static <T> T getData(T value) {
     return value;
  }
  public static void main(String[ ] args) {
     int x = getData(10);
     String s = getData("Vivek");
     System.out.println(x);
     System.out.println(s);
  }
```

```
}
```

Multiple Type Parameters:

```
class Test {
  public static <K, V> void print(K key, V value) {
     System.out.println(key + " : " + value);
  }
  public static void main(String[] args) {
     print(1, "Java");
     print("Age", 25);
  }
}
```

Bounded Types in Java Generics:

Bounded types restrict what types can be used in generics.

Example:

```
class Test<T> {
   T value;
}
Test<String>
Test<Integer>
Test<Object>
```

Imagine You want a method that **only works with numbers** because you want to do math like square, addition, etc.

```
class Test<T> {
  T value;
  void show() {
     // System.out.println(value * value);  Not possible
  }
} Compiler doesn't know what T is, it could be String, Boolean, anything.
```

With Bounded Type :

```
class Test<T extends Number> {
  T value;
  void show() {
    // System.out.println(value * value);
  }
```

```
}
// Now Java knows T will always be a Number.

public class Main {
  public static void main(String[] args) {
    Test<Integer> t1 = new Test<>();
    t1.value = 5;
    t1.show();
    Test<Double> t2 = new Test<>();
    t2.value = 2.5;
    t2.show();
  }
}
Test<String> t3 = new Test<>();//Compile-time error, String is NOT a Number
```

Generics, Inheritance, and Subtypes:

```
class Animal {}
class Dog extends Animal {}
public class Test {
  public static void main(String[] args) {
    Animal a = new Dog(); //  Allowed
  }
}
```

This is **polymorphism** Child IS-A Parent → valid

Generics + Inheritance:

```
import java.util.*;
List<Dog> dogs = new ArrayList<>();
List<Animal> animals = dogs; // Compile-time error
```

Even though Dog extends Animal still NOT allowed Because Generics are INVARIANT

List<Dog> is NOT a subtype of List<Animal>

Generics are invariant in Java, meaning there is no subtype relationship between parameterized types, even if their base types are related.

If it was allowed :

```
List<Dog> dogs = new ArrayList<>();
List<Animal> animals = dogs; // allowed
animals.add(new Animal()); //  Problem
```

Now dogs list contains an Animal (not Dog) breaks type safety.

Wildcards in Java

In Java Generics, wildcards are used when you don't know the exact type. They let you write flexible and reusable code. The wildcard is represented by a ? (question mark). Wildcards are mainly used in method parameters to accept different generic types safely.

Types of wildcards in Java

1. Upper Bounded Wildcards

These wildcards can be used when you want to relax the restrictions on a variable. For example, say you want to write a method that works on List < Integer >, List < Double > and List < Number >, you can do this using an upper bounded wildcard.

To declare an upper-bounded wildcard, use the wildcard character ('?'), followed by the extends keyword, followed by its upper bound.

public static void add(List<? extends Number> list)

Implementation:

```
import java.util.Arrays;
import java.util.List;

class WildcardDemo {
  public static void main(String[ ] args)
  {
     // Upper Bounded Integer List
     List<Integer> list1 = Arrays.asList(4, 5, 6, 7);
```

```
        System.out.println("Total sum is:" + sum(list1));

        // Double list
        List<Double> list2 = Arrays.asList(4.1, 5.1, 6.1);

        System.out.print("Total sum is:" + sum(list2));
    }

    private static double sum(List<? extends Number> list)
    {
        double sum = 0.0;
        for (Number i : list) {
            sum += i.doubleValue();
        }

        return sum;
    }
}
```

Output

Total sum is:22.0

Total sum is:15.299999999999999

Explanation: In the above program, list1 holds Integer values and list2 holds Double values. Both are passed to the sum method, which uses a wildcard <? extends Number>. This means it can accept any list of a type that is a subclass of Number, like Integer or Double.

2. Lower Bounded Wildcards

It is expressed using the wildcard character ('?'), followed by the super keyword, followed by its lower bound: <? super A>.

Syntax: *Collectiontype <? super A>*

Implementation:

```
import java.util.Arrays;
import java.util.List;

class WildcardDemo {
  public static void main(String[] args)
  {
     // Lower Bounded Integer List
     List<Integer> list1 = Arrays.asList(4, 5, 6, 7);

     // Integer list object is being passed
     printOnlyIntegerClassorSuperClass(list1);

     // Number list
     List<Number> list2 = Arrays.asList(4, 5, 6, 7);

     // Integer list object is being passed
     printOnlyIntegerClassorSuperClass(list2);
  }

  public static void printOnlyIntegerClassorSuperClass(
     List<? super Integer> list)
  {
     System.out.println(list);
  }
}
```

Output

[4, 5, 6, 7]

[4, 5, 6, 7]

Explanation: Here, the method printOnlyIntegerClassorSuperClass accepts only Integer or its superclasses (like Number). If you try to pass a list of Double, it gives a compile-time error because Double is not a superclass of Integer.

Note: *Use extend wildcard when you want to get values out of a structure and super wildcard when you put values in a structure. Don't use wildcards when you get and put values in a structure. You can specify an upper bound for a wildcard or you can specify a lower bound, but you cannot specify both.*

3. Unbounded Wildcard

This wildcard type is specified using the wildcard character (?), for example, List. This is called a list of unknown types. These are useful in the following cases -

- When writing a method that can be employed using functionality provided in Object class.
- When the code is using methods in the generic class that doesn't depend on the type parameter

Implementation:

```
import java.util.Arrays;
import java.util.List;

class unboundedwildcardemo {
  public static void main(String[] args)
  {
```

```
        // Integer List
        List<Integer> list1 = Arrays.asList(1, 2, 3);

        // Double list
        List<Double> list2 = Arrays.asList(1.1, 2.2, 3.3);

        printlist(list1);

        printlist(list2);
    }

    private static void printlist(List<?> list)
    {

        System.out.println(list);
    }
}
```

Output

[1, 2, 3]

[1.1, 2.2, 3.3]

Type Erasure in Java

Type Erasure is the process by which the Java compiler removes all generic type information and replaces type parameters with their upper bounds (or Object if unbounded). **Type Erasure** is a core concept in Java Generics that explains *how generics are implemented internally*.

During compilation, the compiler performs:

- Replacing type parameters with their upper bound or Object.

- Inserting casts where needed to maintain type safety.
- Generating bridge methods to preserve method overriding in generic hierarchies.

Note: *After compilation, no generic type exists in the bytecode. Only raw types remain.*

When you write generics like:

List<String> list = new ArrayList<>();

At **compile time**, Java uses `String` for type safety

But at **runtime**, Java **removes (erases)** the generic type

So it becomes:

List list = new ArrayList();

This process is called **Type Erasure**.

Why Java Uses Type Erasure -

Java introduced generics in Java 5, but it needed to:

- Stay compatible with older code (backward compatibility)
- Avoid changing JVM

So instead of adding new runtime behavior, Java:

Checks types at compile time

Removes them at runtime

How Type Erasure Works

1. Replace Generic Types with Object (or Bound)

Example 1:

```
class Box<T> {
   T value;
}
```

After type erasure:

```
class Box {
   Object value;
}
```

2. If Bounded → Replace with Bound

```
class Box<T extends Number> {
   T value;
}
```

After type erasure:

```
class Box {
   Number value;
}
```

3. Add Type Casting Automatically

```
Box<String> box = new Box<>();
String s = box.value;
```

After type erasure:

```
Box box = new Box();
String s = (String) box.value; // compiler adds this cast
```

Type Erasure in Action

Now, we will see examples to understand how type erasure affects code during runtime.

Example 1: Using Generics

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
```

```
public class Demo{

  public static void main(String args[]){

      List<String> list = new ArrayList<>();
      list.add("Hello");

      Iterator<String> iter = list.iterator();
      while (iter.hasNext()) {
          String s = iter.next();
          System.out.println(s);
      }
  }
}
```

Output

Hello

Explanation:

- Compiler checks type safety -> OK (String list)
- At runtime, the generic type (String) does not exist.
- Type casts inserted by the compiler ensure safety.

Example 2: Raw Types

```
import java.util.*;
public class Demo{
   public static void main(String args[]){
      List list = new ArrayList();   // Raw type
      list.add("Hello");
      String s;
```

```
        for (Iterator iter = list.iterator(); iter.hasNext(); ) {
            s = (String) iter.next();
            System.out.println(s);
        }
    }
}
```

Output:

./Demo.java uses unchecked or unsafe operations.

Recompile with -Xlint:unchecked for details.

Explanation: In the above example, we are using raw types, it simply means we are not specifying the type of data the list will hold. In this case, the compiler will throw a warning GenericsErasure.java uses unchecked or unsafe operations. The reason for the warning is that raw types do not carry any type information.

Issues Caused by Type Erasure

The issues caused by Type Erasure is listed below:

- The JVM loses information of the generic type at runtime, that's why we can not use instanceof or create new instances of generic types.
- When we work with raw types, the compiler will give warning.
- We can not create an array of generic types directly. T[] arr = new T[10];
- Method Overloading Issues -
  void print(List<String> list) { }
  void print(List<Integer> list) { }
  After erasure both become:
  void print(List list)

Think of generics like **labels on boxes**:

- At compile time → labeled: `Box<String>`
- At runtime → label removed → just `Box`

Java ensures safety **before execution**, then removes labels.

Tasks to practice on Generics –

1. Generic Class Implementation

Create a generic class `Box<T>` that can store and return a value of any type.

Task:

- Add methods `set(T value)` and `get()`
- Test with `Integer`, `String`, and `Double`

2. Generic Method

Write a generic method that prints elements of any array.

Example:

printArray(new Integer[]{1,2,3});

printArray(new String[]{"A","B","C"});

3. Generic Swap Function

Create a generic method to swap two elements in an array.

4. Count Occurrences

Write a generic method that counts how many times a given element appears in a list.

5. Bounded Type Parameter

Create a generic class `Calculator<T extends Number>` with a method:

double sum(List<T> list)

that returns the sum of elements.

6. Wildcard Usage (? extends)

Write a method:

void printList(List<? extends Number> list)

that prints elements.

7. Wildcard Usage (? super)

Write a method that adds integers to a list:

```
void addNumbers(List<? super Integer> list)
```